

Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

Gwang-Jong Ko^{a,*}, Taesu Cheong^{a,†}, In-Chan Choi^{a,†}

^a*School of Industrial and Management Engineering, Korea University, Seoul, Republic of Korea*

Abstract

A binary linear system (BLS) asks for $\mathbf{x} \in \{0, 1\}^n$ satisfying $A\mathbf{x} = \mathbf{b}$ with $A \in \{0, 1\}^{m \times n}$, a formulation that arises in laser-based non-destructive testing, where a binary occupancy image is reconstructed from integer projections. On market-split-style instances the linear relaxation is deliberately uninformative, so backtracking search spends most of its effort on branching decisions that a cheap heuristic cannot make well. We present a learned branching heuristic for this setting and an exact procedure for training it.

The method rests on two observations. First, conditioning a BLS on a partial assignment is *identical* to substituting the assignment out and obtaining a smaller BLS; a single network therefore predicts at the root and at every interior node without architectural change, and training states are ordinary instances of reduced size. Second, on instances small enough to enumerate, the exact conditional marginal $p_j = \Pr[x_j = 1 \mid A, \mathbf{b}]$ is computable, and it partitions the free variables of a node into those every surviving solution agrees on — where branching the other way empties the subtree and forces a backtrack — and those solutions disagree on, where either branch still reaches a solution. Only the first kind carries cost. This partition supplies both an exact training target and a criterion for which predictions matter: at the root only 6.3% of variables are of the first kind, but by depth 8 the share is 87.9%, which is where prediction accuracy begins converting into search cost.

We train a 21.8K-parameter bipartite graph network on these targets and use it for branching only, never for fixing variables, since it produces a probability rather than a proof. Against LP-probing — the sound procedure that detects the same variables, at one linear program each — the network needs a single relaxation and one forward pass, so their cost ratio grows linearly in instance size: $1.7\times$ at $n = 25$, $7.2\times$ at $n = 60$, $17.8\times$ at $n = 150$, measured with a warm-started backend on both sides. Under a matched wall-clock budget on a warm-started search loop, replicated over three seeds, the resulting search is $2.6\text{--}3.0\times$ faster in median than LP-guided branching at 21×60 (faster on 23/30 instances in every seed, sign test $p = 0.005$) and expands $9.5\times$ fewer nodes, with both strategies solving every instance; within the full pipeline, where lattice reduction runs first, it closes the residual $3.0\times$ faster than LP-guided branching, while random branching closes none of it. Controlling the number of solutions across sizes, we find no measurable transfer penalty: a network trained only at 10×25 is statistically indistinguishable from one trained at the evaluation size. On a 100-instance 21×60 set the supervised guidance is $2.0\text{--}2.3\times$ faster than LP-guided branching with $5.5\times$ fewer nodes. Two further experiments locate what the supervised target does and does not capture. Training on states of deployment size — the regime where the network’s prediction margin lives — does *not* help: it loses competence on the small interior states where most nodes are spent, and the search gets slower. What does help is optimizing the objective the search actually pays: with the exact subtree cost of every branching decision available from the search itself, an actor-critic fine-tuning of the branching policy, initialized at the supervised rule, expands $2.3\times$ fewer nodes and is $2.0\text{--}2.2\times$ faster than the supervised policy on the same 100 instances ($p < 0.01$), $4.8\text{--}5.9\times$ faster than LP-guided branching. The fine-tuned policy branches on a different variable than the confidence rule at 95% of held-out states, typically one it is *less* sure about, which is direct evidence that the probability of being right and the cost of being wrong are different objectives.

We also report what the method does not achieve. CP-SAT solves the same instances $9\text{--}40\times$ faster at every size tested ($9\text{--}10\times$ at 21×60 after fine-tuning), and our system does not compete with it; decomposing that gap shows our branching produces $2.2\times$ fewer nodes but processes them $32\times$ more slowly, which places the deficit in per-node cost rather than in branching quality, though we do not show that it is recoverable. And the cost advantage over probing, while asymptotically real, is only $1.3\text{--}1.7\times$ at the depths where prediction matters most on our instances, where probing additionally offers proofs rather than predictions. The contribution we claim is therefore methodological: exact conditional posteriors are an obtainable and well-posed supervision signal for branching, in-tree states are a better training distribution than root states — a root-trained network is worse than relaxation rounding when deployed in the tree, an in-tree-trained one is better by 3.4 points — and the resulting heuristic transfers across instance size. Code, generators, exact-marginal labels and per-instance results are released.

Keywords: Binary linear systems, Learning to branch, Amortized optimization, Exact conditional marginals, Market split, Graph neural networks

1. Introduction

1.1. Problem

Let $A \in \{0, 1\}^{m \times n}$ and $\mathbf{b} \in \mathbb{Z}^m$. A *binary linear system* (BLS) is the feasibility problem

$$\text{find } \mathbf{x} \in \{0, 1\}^n \text{ such that } A\mathbf{x} = \mathbf{b}, \quad (1)$$

whose solution set we write $\mathcal{S} = \{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}\}$. Each row is a cardinality constraint $\sum_j a_{ij}x_j = b_i$ stating how many cells of a designated subset are occupied.

The motivating application is laser-based non-destructive testing. A specimen is scanned along m directions; each scan returns the integer count of absorbing cells along its path, and the task is to reconstruct the binary occupancy image. Two properties of that setting shape the problem formulation. Solutions need not be unique — several images can be consistent with the same measurements, and recovering *any* member of $\mathcal{S}$ solves the physical problem. And measurement noise can perturb $\mathbf{b}$ so that $\mathcal{S} = \emptyset$, making a reliable infeasibility verdict as valuable as a solution.

Deciding $\mathcal{S} \neq \emptyset$ is NP-complete in general. The instances studied here are drawn from a deliberately hard regime modelled on market-split problems [6, 2], in which the linear relaxation

$$\tilde{\mathcal{S}} = \{\mathbf{x} \in [0, 1]^n : A\mathbf{x} = \mathbf{b}\} \quad (2)$$

is a large polytope with predominantly fractional vertices. Relaxation-based bounding is therefore weak, and a complete solver must decide where to branch with little guidance from the relaxation — exactly the decision this paper learns.

1.2. Where a learned component attaches, and what to train it on

A complete solver for (1) alternates deduction (constraint propagation, relaxation-based pruning) with a *branching decision*: which free variable to split on, and which value to try first. Deduction is sound and must remain so; branching is a heuristic choice, where an error costs search time but never correctness. This asymmetry makes branching the natural place to attach a learned component, and it is where learning has been most successful in mixed-integer programming [9, 7].

The difficulty is supervision. Two standard answers are available and both are unsatisfactory here. Training against a planted solution $\mathbf{x}^*$ treats one member of $\mathcal{S}$ as ground truth, which is ill-posed when $|\mathcal{S}| > 1$: nothing in $(A, \mathbf{b})$ distinguishes the planted solution from any other, so the label is a sample from the posterior rather than the posterior. Imitating an expensive expert, as in strong-branching imitation, requires an expert whose decisions are both informative and affordable to collect; the natural candidate here, LP-probing, costs one linear program per variable per node.

We take a third route, available because these instances have planted solutions and because feasibility can be enumerated

at moderate size: compute the *exact conditional posterior* by exhaustive enumeration on small instances, and train against it. Section 3 shows that this is both tractable and, in a precise sense, the right target — and that the posterior additionally identifies *which* predictions can affect search at all.

1.3. Contributions

1. **A reduction identity that makes in-tree supervision trivial to implement.** Conditioning (1) on a partial assignment yields a smaller BLS with the same structure (Proposition 1). One architecture, one label generator and one training loop therefore serve the root and every interior node, and a depth- d training state is stored as an ordinary instance on $n - d$ variables.
2. **A partition of branching decisions by whether they can cost anything.** The exact marginals split a node’s free variables into DET (all surviving solutions agree; branching the other way empties the subtree) and OPEN (solutions disagree; either branch reaches a solution). Only DET errors cost a backtrack. We measure the composition: 6.3% DET at the root, rising to 87.9% at depth 8 (Sections 4.3, 4.4). This locates useful prediction inside the tree rather than at the root and explains why root-level accuracy is a poor objective.
3. **A cost characterization of the learned heuristic against the sound procedure it approximates.** The network needs one relaxation and one forward pass; LP-probing needs one relaxation per variable. Measured with a warm-started backtrack on both sides, the ratio scales from $1.7\times$ at $n = 25$ to $17.8\times$ at $n = 150$ (Section 4.5). We report this as a scaling law rather than a single figure, and note that at the sizes where our search operates the advantage is modest and probing has the additional merit of soundness.
4. **End-to-end evaluation under matched wall-clock budgets.** At 21×60 on a warm-started search loop, replicated over three seeds, the learned guidance is $2.0\text{--}2.3\times$ faster in median than LP-based branching on a 100-instance set (faster on 70–71/100, sign test $p < 0.001$) and expands $5.5\times$ fewer nodes; on the original 30-instance set the margins are $2.6\text{--}3.0\times$ and $9.5\times$, and both strategies solve every instance within 600 s (Section 4.6). In the deployed cascade both sound strategies close every residual instance, the learned one $3.0\times$ faster (40/50 wins pooled over seeds, $p = 2 \times 10^{-5}$), while random branching closes none of the 50 residual instances (Section 4.12).
5. **Size transfer with solution multiplicity controlled.** Holding $|\mathcal{S}|$ comparable across sizes, a network trained only at 10×25 matches one trained at the evaluation size 21×60 (faster on 15/30 instances; total time differing by 0.4%), showing the learned quantity is not size-specific (Section 4.8).
6. **A negative result on aligning the training distribution with deployment.** The network’s margin over relaxation rounding lives on states with 50–60 free variables, outside its 13–25 training range. Training on exactly those states, with everything else fixed, raises accuracy there by about

*Corresponding author.

†Footnote text for the † mark to be supplied by the authors.

one point and *doubles* the search tree, because it loses competence on the small interior states where two thirds of all nodes are spent (Section 4.10).

7. **Cost-aware fine-tuning of the branching policy.** The exact cost of a branching decision — the number of nodes its subtree expands — is available from the search itself, and on held-out states the confidence rule picks the cheapest of its five candidates only 22% of the time. An actor-critic fine-tuning on that cost, initialized at the supervised rule with the marginal network frozen, expands 2.3× fewer nodes and is 2.0–2.2× faster than the supervised policy on the 100-instance set (70/100 and 64–67/100 wins, $p < 0.01$ in every seed), 4.8–5.9× faster than LP-guided branching (Section 4.11).

2. Related Work

2.1. Binary linear systems and market-split hardness

Problem (1) is a 0/1 integer feasibility problem and is NP-complete for general A . Practical hardness, however, depends on the instance distribution. Cornuéjols and Dawande [6] introduced the market-split family, in which m agents must divide n items so that every agent’s budget is met exactly; with $n \approx 10(m - 1)$ these instances defeat branch-and-bound at remarkably small sizes. The mechanism is the integrality gap: the relaxation polytope is large and its vertices are fractional, so bounding prunes little and the search must branch nearly blindly. Aardal et al. [2] showed that lattice basis reduction attacks the same family far more effectively than branch-and-bound, a result we rely on in Section 3.2.

Our generator (Section 4.1) makes this mechanism explicit by maximizing the spread of relaxation vertices, which enlarges $\tilde{S}$ and weakens the relaxation signal by construction.

2.2. Complete solvers

Mixed-integer programming. Branch-and-bound over the linear relaxation, strengthened by presolve and cutting planes, is the standard approach. Its leverage comes from pruning nodes whose relaxation is infeasible or bounded away from the target, which is precisely what the market-split construction removes.

Constraint programming with clause learning. CP-SAT-style solvers combine constraint propagation with conflict-driven clause learning [12, 14] and lazy clause generation [15]. Their strength on this family comes from a property central to our argument: a search *manufactures* information as it descends. A marginal $\Pr[x_j = 1 \mid A, \mathbf{b}]$ may be uninformative at the root while $\Pr[x_j = 1 \mid A, \mathbf{b}, x_1=1, x_5=0]$ is 0 or 1. Structure invisible at the root becomes deducible after commitments are made, which is why Section 3.4 studies the interior of the tree rather than the root.

Lattice reduction. An alternative attack embeds (1) in a lattice and searches for short vectors. LLL [10] reduces a basis in polynomial time and BKZ [16] generalizes it with a block size trading time against basis quality. The Aardal–Hurkens–Lenstra

embedding [1, 2] is constructed so that a 0/1 solution appears directly as a short vector, making reduction a certificate-producing procedure rather than a heuristic. We use it as the first stage of the deployed pipeline. Lattice enumeration remains the state of the art on the canonical benchmark: Wassermann [35] solves QOBLIB market-split instances up to $m = 14$ on one CPU where branch-and-cut stops at $m = 7$; our instances are a different regime (multiplicity matched, $m/n \approx 0.35$) and our contribution is a branching methodology rather than a record there.

2.3. Learning for combinatorial optimization

Bengio et al. [4] survey this field and frame it in terms we adopt: solvers rely on hand-crafted heuristics for decisions that are “otherwise too expensive to compute or mathematically not well defined,” and learning is a candidate for making those decisions better. The phrase *too expensive to compute* presumes the decision is computable in principle and asks only about cost, which is the situation we measure in Section 4.5.

Marginals of the solution set as a branching signal. The fraction of solutions agreeing with a decision was a branching criterion before learning: counting-based search [32] branches on the variable–value pair of maximum solution density computed inside individual constraints, and belief-propagation-guided decimation [29] fixes the most biased variable under approximate marginals recomputed after each fixing. Learned versions replace the counting by a network: NSNet [28] trains a BP-structured graph network on exact marginals of a SAT formula’s satisfying assignments, obtained by enumeration, and rounds them into a local-search initialization; MIP-GNN [27], Neural Diving [30] and predict-and-search [26] predict per-variable biases from collected near-optimal solutions and use them to fix variables, warm-start or restrict a search. Our target is the same as NSNet’s; what differs is that the marginals are conditional, carried to every node by the reduction identity, and used to guide a complete search without fixing anything, and that the DET/OPEN partition and the in-tree-versus-root comparison have no counterpart there.

Branching imitation. The clearest success in this area is imitation of an expensive expert. Strong branching scores candidates by tentatively solving both children’s relaxations; it yields small trees and is too slow to run at every node. Gasse et al. [7] train a bipartite graph network to imitate it and recover most of the benefit at a fraction of the cost, following Khalil et al. [9]. The gain is not new information — the expert already knew the answer — but cheap access to it. Our method occupies the same structural position, with two differences: the target is an exact posterior rather than an expert’s scores, and the analysis identifies which of the expert’s decisions matter.

Direct satisfiability prediction. Attempts to have a network decide satisfiability end-to-end have a weaker record. NeuroSAT [17] classifies satisfiability by message passing over a literal–clause graph and succeeds on small random instances, but Li et al. [11] report substantial accuracy collapse under size transfer.

Chen et al. [5] prove that standard message-passing networks cannot separate certain feasible/infeasible MILP pairs at all, since the pairs are indistinguishable in the Weisfeiler–Lehman hierarchy that bounds such architectures [36]. We therefore do not ask the network for a feasibility verdict; it supplies branching guidance inside a complete search that remains responsible for soundness.

Graph representations. We use graph attention [18] over the bipartite variable–constraint graph, the standard encoding for MILP in this literature [7].

Reinforcement learning for branching. Imitation targets an expert’s ranking, not the search cost the ranking is meant to reduce. FMSTS [20] uses a node’s subtree size as an observable Q-value, shows that under depth-first search minimizing every subtree minimizes the whole tree, and learns from scratch; tree MDPs [21] formalize that per-subtree credit assignment; retro branching [31] learns from retrospectively extracted subtree trajectories; SORREL [25] initializes from suboptimal demonstrations; Neuro# [34] trains a graph network on the residual formula of a #SAT solver by evolution strategies to minimize branching decisions. In constraint programming, value ordering has been learned from solutions [24] and by deep Q-learning [23], and impact-based search [33] prefers the variable whose assignment shrinks the search space most. Our fine-tuning stage (Section 3.8) uses the same per-subtree credit with a plain policy-gradient update [19], and differs in its starting point: the policy is initialized at a supervised rule derived from exact conditional marginals, and the marginal network is frozen, so the only thing the cost signal can change is which variable is branched on. The classical statement of the objective is the fail-first principle [22]: choose the variable whose wrong branch is refuted soonest.

2.4. Amortized optimization

The distinction organizing our results predates its use in combinatorial optimization. Gershman and Goodman [8] introduced *amortized inference*: when many related queries must be answered, solving each from scratch is wasteful and the question becomes how to reuse computation across queries. Amos [3] develops the same idea as *amortized optimization* — using learning to predict solutions by exploiting shared structure across similar instances — and surveys its appearance in variational inference, meta-learning and control. Millidge [13] states the trade-off in the form most relevant here: *direct* optimization spends computation on the instance at hand and pays that cost at every invocation, whereas *amortized* optimization converts the problem into supervised learning with cheap inference but requires a corpus of solved instances up front and is bounded by the approximator’s generalization; the productive arrangement is hybrid, with amortized predictions steering a direct optimizer.

Our system instantiates that hybrid precisely. The direct method is LP-probing, sound and costly. The amortized method is the network, cheap and unsound. The hybrid rule is that the network steers branching while probing and propagation retain exclusive authority over variable fixing (Section 3.7).

3. Proposed Method

3.1. Setting and notation

For a partial assignment fixing variables $F \subseteq \{1, \dots, n\}$ to values $\mathbf{v} \in \{0, 1\}^{|F|}$, write

$$\mathcal{S}(F, \mathbf{v}) = \{\mathbf{x} \in \mathcal{S} : \mathbf{x}_F = \mathbf{v}\} \quad (3)$$

for the solutions consistent with it, and let $K = \{1, \dots, n\} \setminus F$ index the free variables. The *conditional marginal* of a free variable $j \in K$ is

$$p_j(F, \mathbf{v}) = \frac{|\{\mathbf{x} \in \mathcal{S}(F, \mathbf{v}) : x_j = 1\}|}{|\mathcal{S}(F, \mathbf{v})|}. \quad (4)$$

3.2. System overview

The deployed solver is a cascade of a lattice stage and a guided complete search:

$$\begin{aligned} \text{propagation} &\rightarrow \text{LP infeasibility rule} \rightarrow \text{kernel pump} \\ &\rightarrow \text{AHL/BKZ} \rightarrow \text{guided backtracking search.} \end{aligned} \quad (5)$$

The first three stages are cheap and sound. AHL/BKZ attempts a certificate-producing lattice reduction with a size-dependent block parameter; when it succeeds it returns a verified solution outright. The learned component appears only in the final stage, and only in its branching decisions. This placement matters for the evaluation: because the lattice stage is independent of the branching heuristic, every heuristic under comparison receives exactly the same residual set of instances, so conditioning on that residual describes the algorithm rather than selecting favourable cases.

3.3. Conditioning is reduction

Proposition 1. Let A_F and A_K denote the column submatrices of A indexed by F and K . Then

$$\mathcal{S}(F, \mathbf{v}) \cong \{\mathbf{y} \in \{0, 1\}^{|K|} : A_K \mathbf{y} = \mathbf{b} - A_F \mathbf{v}\}, \quad (6)$$

the bijection being $\mathbf{x} \mapsto \mathbf{x}_K$. Consequently the conditional marginals (4) of the original instance are the ordinary marginals of the reduced instance $(A_K, \mathbf{b} - A_F \mathbf{v})$.

Proof. $A\mathbf{x} = A_F \mathbf{x}_F + A_K \mathbf{x}_K$. Fixing $\mathbf{x}_F = \mathbf{v}$ gives $A\mathbf{x} = \mathbf{b} \iff A_K \mathbf{x}_K = \mathbf{b} - A_F \mathbf{v}$, and $\mathbf{x} \mapsto \mathbf{x}_K$ is a bijection between the two sets since $\mathbf{x}_F$ is determined. The marginal statement follows by counting. $\square$

The identity is elementary but it is what makes in-tree supervision practical. A depth- d search node is not a special object requiring its own encoding; it is an instance of the same problem on $n - d$ variables. One network, one feature extractor and one label generator therefore cover the whole tree, and training data at depth d is produced by substitution rather than by instrumenting a solver.

Two implementation details preserve correctness. Rows that become all-zero after substitution carry no constraint and are removed; retaining them causes linear-programming backends to reject the model. And the free bound condition $0 \leq \mathbf{b} - A_F \mathbf{v} \leq A_K \mathbf{1}$ is checked before any relaxation is solved, since it detects a large fraction of dead nodes at negligible cost.

3.4. Which predictions can cost anything

At a node with surviving solution set $\mathcal{S}(F, \mathbf{v}) \neq \emptyset$, partition the free variables by their conditional marginal:

$$\text{DET} = \{j \in K : p_j \in \{0, 1\}\}, \quad \text{OPEN} = \{j \in K : 0 < p_j < 1\}. \quad (7)$$

The distinction is operational rather than statistical:

- For $j \in \text{OPEN}$, both children of a branch on x_j contain solutions. Either choice keeps a solution reachable, so *no branching decision on a OPEN variable can cost anything*.
- For $j \in \text{DET}$, one child is empty. Branching against the forced value guarantees that the subtree is exhausted without a solution, costing a full backtrack.

This gives the conditional marginal its operational reading. It is not a “probability of being correct” about some designated solution, but the *fraction of the surviving solution set retained* by an assignment; and the only errors that matter are those on variables where that fraction is 0 or 1.

Two consequences follow, both of which the experiments confirm. First, an aggregate accuracy over all free variables is a poor objective, because it mixes decisions that can cost a backtrack with decisions that cannot; the relevant quantity is accuracy on DET. Second, since $|\mathcal{S}(F, \mathbf{v})|$ shrinks as the search descends, the DET share grows with depth. This alone does not say *where* a learned predictor is useful, because a DET variable that the relaxation already rounds correctly needs no predictor. The useful region is where DET variables are plentiful *and* the relaxation is weak, and Section 4.9 shows that this is governed by constraint density $m/|K|$ rather than by depth as such: on small instances the two coincide, on larger ones they do not.

3.5. Exact conditional marginals as training targets

For instances small enough to enumerate, $\mathcal{S}$ is obtained exactly with a constraint solver in all-solutions mode, and (4) is computed by counting. Training minimizes the cross-entropy to the resulting Bernoulli targets,

$$\mathcal{L} = -\frac{1}{|K|} \sum_{j \in K} [p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j)], \quad (8)$$

which is minimized exactly at $\hat{p}_j = p_j$. Note that the same loss with a planted solution in place of p_j — the conventional choice — is a noisy surrogate whenever $|\mathcal{S}| > 1$, since the planted solution is one draw from the posterior; on our instances a Bayes-optimal predictor agrees with it only about two thirds of the time.

Enumeration correctness. A subtlety here is easy to get wrong and silently corrupts every label derived from it. Constraint solvers report distinct statuses for “search exhausted” and “time limit reached with solutions already collected.” Only the former certifies that $\mathcal{S}$ is complete; accepting the latter substitutes a truncated solution set, which biases every marginal computed from it. In our implementation only OPTIMAL and INFEASIBLE are accepted and timed-out instances are discarded rather than

averaged in. A useful diagnostic is that mean enumeration time equal to the time limit indicates truncated instances being counted as complete.

Sampling reachable states. A depth- d training state is built by drawing d uniformly from $\{0, \dots, 12\}$, taking a prefix of an LP-confidence variable ordering, and fixing those variables to the values of a randomly chosen *actual solution*. Fixing arbitrary values would produce states with $\mathcal{S}(F, \mathbf{v}) = \emptyset$; such states are dead nodes that propagation prunes immediately and on which the conditional marginal is undefined, so they are not states a branching heuristic is ever asked about. The depth range is a design choice we did not tune; on 10×25 instances it yields training states with 13–25 free variables, and Section 4.9 examines what happens when the deployed search queries states outside that range, which at 21×60 is most of them.

3.6. Network

An instance is encoded as a bipartite graph with n variable nodes, m constraint nodes, and an edge (j, i) whenever $a_{ij} = 1$. Let $\tilde{\mathbf{x}}$ be the relaxation solution of the current (reduced) instance and $r_i = \sum_j a_{ij}$ the row sums. Node features are

$$\text{variable } j : (\tilde{x}_j, \deg(j)/m, |\tilde{x}_j - \text{round}(\tilde{x}_j)|), \quad (9)$$

$$\text{constraint } i : (b_i/r_i, r_i/n, (b_i - A_i\tilde{\mathbf{x}})/r_i), \quad (10)$$

i.e. relaxation value, column degree and fractionality for variables; tightness, scaled row size and relaxation residual for constraints. All are size-normalized, which is what permits a single parameter set to apply at any (m, n) .

Both node types are embedded to $h = 64$ dimensions by a linear map and refined by $L = 4$ rounds of bidirectional message passing with graph attention [18]:

$$\mathbf{h}^{\text{con}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{var}} \rightarrow \mathbf{h}^{\text{con}})) + \mathbf{h}^{\text{con}}), \quad (11)$$

$$\mathbf{h}^{\text{var}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{con}} \rightarrow \mathbf{h}^{\text{var}})) + \mathbf{h}^{\text{var}}), \quad (12)$$

followed by a two-layer head emitting one logit per variable, $\hat{p}_j = \sigma(z_j)$. The model has 21,761 parameters.

The architecture is deliberately small and single-headed. Its only output is the quantity the objective (8) defines; there are no auxiliary value or feasibility heads, because the search draws soundness from its symbolic components and asks the network for nothing else.

3.7. Inference: guidance, not commitment

The network is consulted once per node. Branching selects

$$j^* = \arg \max_{j \in K} |\hat{p}_j - \frac{1}{2}|, \quad \text{first value} = \mathbb{K}[\hat{p}_{j^*} \geq \frac{1}{2}], \quad (13)$$

that is, the variable the network is most confident about, tried at its predicted value. The search remains complete and a wrong prediction costs backtracking only.

Variables are *fixed* only by sound deduction. Two mechanisms are available: constraint propagation, and LP-probing, which fixes x_j when assigning the opposite value makes even the relaxation infeasible. The division of labour follows directly

from the measurements in Section 4.5: where a wrong answer costs time, the cheap unsound predictor is preferable; where a wrong answer costs correctness, the expensive sound procedure is required and cannot be replaced.

Algorithm 1 states the resulting search.

Algorithm 1 Guided backtracking search for a BLS

Input: instance $(A, \mathbf{b})$, guidance model f_θ , time budget T
stack $\leftarrow [(A, \mathbf{b})]$
while stack $\neq \emptyset$ **and** elapsed $< T$:
 $(A', \mathbf{b}') \leftarrow \text{pop}(\text{stack})$
 if $\mathbf{b}' < \mathbf{0}$ **or** $\mathbf{b}' > A' \mathbf{1}$: **continue** // free bound check
 drop all-zero rows of A' ; **if** none remain: **return** any completion
 propagate; **if** contradiction: **continue**
 if propagation determined a set D of variables:
 push reduce($A', \mathbf{b}', D$); **continue** // Proposition 1
 if all variables assigned: **return** solution if verified, else **continue**
 $\hat{\mathbf{p}} \leftarrow f_\theta(A', \mathbf{b}')$ // one forward pass, all variables
 $j^* \leftarrow \arg \max_j |\hat{p}_j - 1/2|$; $v \leftarrow \mathbb{I}[\hat{p}_{j^*} \geq 1/2]$
 push reduce($A', \mathbf{b}', \{j^* = 1-v\}$), then reduce($A', \mathbf{b}', \{j^* = v\}$)
return unresolved

3.8. Cost-aware fine-tuning of the branching policy

The rule of Algorithm 1 branches on the variable the network is surest about. That maximizes the probability that the first child contains a solution; it says nothing about how much the search pays when the first child does not, nor about how much smaller the children are. Both quantities are properties of the *subtree*, and a search produces them as a by-product: the cost of a branching decision at node u is the number of nodes expanded below u until a solution is found there or the subtree is exhausted, and depth-first search returns it exactly when it backtracks out of u . We use this cost to fine-tune which variable is branched on.

Decision process. A state is the reduced instance $(A', \mathbf{b}')$ after propagation; an action is a free variable j ; the first value tried remains $\mathbb{I}[\hat{p}_j \geq 1/2]$ from the marginal network, which is not modified. The return of the action at u is $-\log c(u)$ with $c(u)$ the subtree cost. Because every decision has its own subtree, credit is assigned per decision rather than per episode, which is what made the sparse-reward formulations we tried earlier unlearnable.

Policy and value. On the frozen MarginalNet trunk we add two heads. The policy logit is

$$g_j(s) = \log \left| \hat{p}_j - \frac{1}{2} \right| + h_\phi(\mathbf{h}_j), \quad (14)$$

where h_ϕ is a two-layer network on the variable embedding $\mathbf{h}_j$ whose last layer is initialized to zero, so that $\pi(j | s) \propto \exp(g_j/\tau)$ starts as a softening of the supervised rule and its argmax is exactly Algorithm 1. The value head $V_\psi(s)$ reads a pooled graph embedding together with $\log |K|$ and $\log m$ and predicts $\log c$.

Update. Each epoch samples training states, rolls the sampled policy ($\tau = 1$) out to a solution, and records every decision with its exact cost. With advantage $\alpha(u) = V_\psi(s_u) - \log c(u)$, standardized within the epoch, the update is

$$\mathcal{L} = \sum_u \left[-\alpha(u) \log \pi(j_u | s_u) - \lambda \mathcal{H}(\pi(\cdot | s_u)) + (V_\psi(s_u) - \log c(u))^2 \right], \quad (15)$$

with entropy weight $\lambda = 0.01$. Only ϕ and ψ are trained; the trunk and the marginal head are fixed, so the marginal used for the first value and for the analysis of Section 3.4 is unchanged. The critic is pre-trained on decisions recorded under the supervised policy before the actor is updated, and its held-out rank correlation with the true cost is used as a gate: if the baseline cannot rank costs, the advantage is noise and fine-tuning is not attempted. Epoch selection uses the total subtree cost on held-out training-distribution states, never the evaluation instances. At deployment the policy is used greedily: $j^* = \arg \max_j g_j(s)$.

3.9. Cost model

The comparison that motivates the design is per-node cost for a whole-instance verdict, and its structure is asymptotic rather than a constant factor. Let n_c be the number of free variables at a node. The network solves *one* relaxation to build its features and performs *one* forward pass, so its LP count is $O(1)$ in n_c . LP-probing solves one relaxation per candidate variable, $O(n_c)$. The ratio between them therefore grows linearly in instance size, and any single number quoted for it is meaningful only together with the n_c at which it was measured.

This also means the comparison is sensitive to how the LP backend is implemented, in a way that cuts against the learned method. A backend that rebuilds the model for every call inflates probing's cost by its full $O(n_c)$ factor while charging the network only once, exaggerating the advantage. A correctly engineered branch-and-bound keeps one relaxation and re-optimizes from the parent basis after a bound change, which is an order of magnitude cheaper per probe. Section 4.5 measures both methods against a warm-started backend for this reason, and reports the ratio as a function of n_c rather than as a single figure.

4. Experiments

4.1. Instances and control of solution multiplicity

Instances are generated by the procedure GENHARD(m, n, K): draw a random 0/1 matrix A of density $1/2$; plant $K = 20$ candidate solutions $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(K)}$ with $\mathbf{b}^{(k)} = A\mathbf{x}^{(k)}$; retain the one maximizing *vertex spread*, the mean pairwise L_2 distance among relaxation vertices obtained from random objective directions. Maximizing vertex spread enlarges $\hat{S}$ directly and is the mechanism that makes market-split instances hard for relaxation-based methods (Section 2). Uniqueness of the planted solution is not enforced.

Why $|\mathcal{S}|$ must be controlled across sizes.. Solution multiplicity changes the *character* of the prediction task, not only its difficulty. When $|\mathcal{S}| = 1$ every free variable is `DET` and predicting well is tantamount to solving; when $|\mathcal{S}|$ is large most variables are `OPEN` and the task is to estimate a diffuse posterior. Comparing sizes without controlling $|\mathcal{S}|$ therefore confounds size with task identity. Since $|\mathcal{S}|$ falls as m rises at fixed n , we select m per size to match it.

Limits of the control.. Matching $|\mathcal{S}|$ forces m/n to vary ($0.40 \rightarrow 0.35$); at fixed n the two cannot be held simultaneously and we prioritize $|\mathcal{S}|$ because it determines task identity. The control is also granular: at $n = 60$ the median $|\mathcal{S}|$ steps $42 \rightarrow 12 \rightarrow 2$ across $m = 20, 21, 22$, so $m = 21$ is the best available integer and the realized evaluation set has median $|\mathcal{S}| = 10$ with 27/30 verified.

Where enumeration stops.. At $n = 70$ no enumeration completed within 120 s at the m giving the target multiplicity (0/10). At $n = 100$ the obstruction is more basic: a constraint solver finds *zero* solutions within 20 s at $m \in \{28, 32, 36, 40\}$ despite a planted one. At that size the bottleneck is finding any solution rather than choosing branches well, so a branching comparison there would not be informative. We therefore evaluate at $n \leq 60$ and report this boundary as a limitation.

4.2. Implementation and protocol

Software.. Relaxations used for training features and for the original search loop are solved with SCIP via PySCIPOpt; the warm-started relaxations of Sections 4.5 and 4.6 use Gurobi 13 (dual simplex, bounds changed in place), which requires a licence — HiGHS exposes an equivalent incremental interface and should serve as a free substitute, though we have not verified the timings with it. Enumeration and the CP-SAT baseline use OR-Tools; lattice reduction uses fpylll (LLL and BKZ). The network is implemented in PyTorch with PyTorch Geometric (GATConv). One property of the relaxation deserves note: it carries no objective, so its solution is an arbitrary vertex of the relaxation polytope and different LP codes return different vertices. Both branching arms consume the same vertex within an experiment, so comparisons are fair, but absolute node counts differ between the SCIP- and Gurobi-backed runs.

Training.. Adam, learning rate 10^{-3} , 20 epochs over 10,000 in-tree states, batch size 1 (instances vary in size), gradient-norm clipping at 1.0, loss (8) with exact marginal targets. Training states are drawn at depths 0–12 uniformly, 6 states per source instance, from a pool of 1,760 instances; evaluation uses 440 instances from a disjoint pool. Training takes approximately 20 minutes on one CPU core. *No hyperparameter was tuned:* width, depth, learning rate, epoch count and the training depth range are the values first tried, and there is no validation split. Reported figures are therefore not contaminated by selection on the test set, but neither is any of these settings claimed to be optimal.

Threading.. All timing-sensitive components run single-threaded (`torch.set_num_threads(1)`). On graphs of this size multi-threaded BLAS is counterproductive: we measure 11.3 ms per forward pass at 8 threads against 0.9 ms at one.

Search evaluation.. Each arm is run on identical instances with a hard per-instance wall-clock budget enforced by subprocess termination, since lattice reduction executes inside a blocking native call that in-process timers cannot interrupt. Instances run at concurrency 6 on a 20-core machine. This low concurrency is deliberate: wall-clock-budgeted comparisons are sensitive to CPU contention, and at higher concurrency we have observed instances solvable in 14–15 s standalone exceeding a 20 s cap.

Statistics.. Solve-rate differences between arms on the same instances are tested with McNemar’s exact test; per-instance speed differences with a two-sided sign test. We report both rather than a single aggregate because they can have different power. The main comparison at 21×60 is replicated over three independent seeds and we report the spread; smaller-size and transfer results are single-run and marked as such. Seeds are set explicitly: an earlier version derived them from `hash()` of the instance filename, which Python randomizes per process, so those runs had no recorded provenance.

Evaluation sets.. Two 21×60 sets are used: the original 30 instances (generator seeds 700000–700029) and a 100-instance set (seeds 700000–700099) that contains them, built after the 30-instance results had been reported; $|\mathcal{S}|$ is verified for 92/100, median 11. All numbers on the 100-instance set are lattice-off, three seeds. The training pool for the experiments of Sections 4.10 and 4.11 is a separate set of 500 instances at 21×60 (seeds 800000–800499, split 400/100) whose overlap with every evaluation set is zero by hash of $(A, \mathbf{b})$; the 100 held-out instances of that pool supply the states on which fine-tuning epochs are selected.

4.3. Prediction quality against the Bayes ceiling

Because $\mathcal{S}$ is enumerated exactly, the Bayes-optimal per-variable accuracy is computable:

$$\text{ceil} = \frac{1}{|K|} \sum_{j \in K} \max(p_j, 1 - p_j), \quad (16)$$

attained by answering the majority value of the surviving solutions. No deterministic predictor of $\mathbf{x}^*$ from $(A, \mathbf{b})$ can exceed it. Table 2 evaluates predictors against it on 400 held-out 10×25 instances with $\mathcal{S}$ fully enumerated.

The network comes within 4.1 points of the ceiling overall and 6.6 on the top-3 subset, and it recovers most of the gap between relaxation rounding and the ceiling ($60.9 \rightarrow 66.7$ against a ceiling of 70.8). Two preliminary experiments on an earlier pool of 10×25 instances, reported for completeness but not from the checkpoint evaluated here, bear on design choices: a 2.11M-parameter network with four output heads (selection, assignment, value and feasibility) scored 60.7% on root states, statistically indistinguishable from relaxation rounding

Size	m	m/n	median $ S $	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

Table 1: Sizes used, with solution multiplicity matched. Ceilings (the Bayes-optimal per-variable accuracies of Section 4.3) agree to within one point between the two training sizes, so size is the variable that changes.

Predictor	all variables	top-3	L_1 to true marginal
Bayes ceiling	70.8%	93.2%	0.0000
Proposed network	66.7%	86.6%	0.1216
LP relaxation rounding	60.9%	65.8%	—

Table 2: Root-level prediction against the exact posterior on the 182 root states of the held-out 10×25 evaluation pool. The network is the same checkpoint used in every search experiment below. “top-3” restricts to the three variables the predictor is most confident about.

Root variables	count	share	ceiling on subset
DET (branching against it empties the subtree)	38	6.3%	100.0%
OPEN (either branch reaches a solution)	562	93.7%	67.2%
all variables	600	100%	69.3%

Table 3: Root-level decomposition, 24 instances of 10×25 with S enumerated. The aggregate is 94% composed of variables on which no branching decision can cost a backtrack.

at 60.6%, so capacity is not the binding constraint and a single-purpose head is preferable; and training the single-head architecture on exact marginals rather than the planted solution improved L_1 to the posterior from 0.1261 to 0.0977 while leaving argmax accuracy unchanged, which matters because the analysis of Section 3.4 consumes the marginal rather than its argmax.

What the aggregate conceals.. Table 3 decomposes the same root variables by the partition of Section 3.4.

At the root, 93.7% of variables are OPEN. The aggregate ceiling of 69.9% is therefore dominated by predictions that cannot affect the search, and on the 6.3% that can, the information is already complete (100%). Root-level accuracy is consequently a weak objective for a branching heuristic, and the operative question is not whether the information exists but what detects it — taken up next.

4.4. Depth, and where the residual lies

Using Proposition 1 we measure, at each depth along a prefix drawn from an actual solution, the conditional ceiling and the fraction of remaining variables that propagation forces (Table 4).

The consequential movement is in composition rather than level: the DET share rises from 6.3% at the root to 87.9% at depth 8 (Table 6). At the root almost every branching decision is inconsequential because both children contain solutions; by depth 8 almost every decision empties a subtree when wrong. The network’s margin over relaxation rounding is 3–6 points

throughout and does not vanish at depth, which is the first indication that what it captures is not simply depth-dependent; Section 4.9 makes this precise.

Does in-tree supervision matter, controlling for everything else?. A comparison between a root-trained and an in-tree-trained network is only informative if nothing else differs. Table 5 holds the instance pool, the label type (exact marginals), the architecture, the optimizer and the number of training states (1,760 each) fixed, and varies only whether the states are root states or in-tree states drawn at depths 0–12. Both are evaluated on the same held-out in-tree states.

In-tree supervision is worth 4.4 points overall. The more telling row is the root-trained network itself: at 75.4% it is *below* relaxation rounding on in-tree states, so a network trained on root states does not merely underperform in the tree, it is worse than the classical heuristic it is meant to replace. The shift from root to interior states is not a distribution the root-trained network generalizes across, and in-tree training is what makes the learned guidance better than the baseline at all.

Table 6 decomposes the residual gap at these depths and asks what each method detects.

The gap lies entirely on DET variables: at depth 7, $0.80 \times 11.2 \approx 9$ points, matching the measured 10.3-point shortfall against the conditional ceiling. Performance near 50% on OPEN variables is correct behaviour rather than a defect, since those variables are genuinely undetermined. The striking figure is that unit propagation detects only 1.2–5.3% of logically determined variables: the information is present and derivable but not reachable by the cheapest sound rule. LP-probing recovers 52–80% and LP integrality 68–77%.

4.5. Cost per node

Both methods are charged everything they must pay to reach a whole-instance verdict: the network one relaxation for its features plus one forward pass, probing one relaxation per free variable. Both use the same warm-started backend, a single model whose variable bounds are changed in place and re-optimized from the parent basis (Gurobi dual simplex), which is how a branch-and-bound implementation reuses its relaxation. Timings are single-threaded.

The ratio behaves as the cost model predicts: it is negligible at $n = 25$ and reaches $17.8\times$ at $n = 150$, tracking the $O(n_c)$ versus $O(1)$ difference in relaxation count. Two consequences follow, and they point in opposite directions.

The advantage is real but asymptotic, and small where we can search.. At the sizes where our search actually operates

depth	surviving $ \mathcal{S} $	conditional ceiling	proposed network	LP rounding
0	25.3	70.8%	66.7%	60.9%
4	4.2	82.3%	72.8%	68.5%
7	1.8	91.5%	81.2%	77.8%
8	1.4	94.4%	84.3%	80.9%
12	1.1	98.8%	96.8%	95.5%
all depths	—	87.2%	80.0%	76.5%

Table 4: Conditional information against depth on the held-out in-tree states of the 10×25 evaluation pool ($n = 25$), LP-confidence variable ordering. Same checkpoint as Table 2.

depth	conditional ceiling	root-trained	in-tree-trained	LP rounding
0	70.7%	67.8%	66.5%	60.9%
4	82.1%	71.4%	72.4%	68.3%
6	89.7%	74.6%	78.2%	74.0%
8	94.4%	77.4%	84.2%	81.3%
10	97.1%	81.7%	91.4%	89.4%
12	98.8%	85.8%	96.5%	95.3%
all depths	87.2%	75.4%	79.8%	76.4%

Table 5: Controlled comparison of training-state distribution. Same pool, labels, architecture, optimizer and state count; only the depth at which states are sampled differs. Evaluated on held-out in-tree states of the 10×25 pool.

($n = 25$ – 60) the ratio is 1.3 – $7.2\times$. At the depths where prediction matters most (5 – 8 of a 10×25 instance, $n_c = 17$ – 20) it is 1.3 – $1.7\times$. A practitioner choosing between the two at that scale should prefer probing, which is *sound*: where its test fires the conclusion is a proof, whereas the network never offers a guarantee. The learned alternative becomes compelling only at $n \gtrsim 100$, and Section 4.1 shows that search itself fails there for every strategy we tried.

The measurement is sensitive to baseline engineering... An earlier version of this comparison used a backend that rebuilt the LP model on every call and, additionally, excluded the network’s own feature relaxation from its timer. That combination reported 19 – $23\times$ at depths 5 – 8 , where the correctly measured value is 1.3 – $1.7\times$ — an overstatement of more than an order of magnitude, arising entirely from how the two sides were instrumented rather than from what they compute. We report the corrected figures and note the general hazard: in any cost comparison against a classical procedure, the classical side must be implemented as a practitioner would implement it, and the learned side must be charged for its own preprocessing.

4.6. Search performance

We compare branching strategies under matched wall-clock budgets with the lattice stage disabled, isolating branching quality. LP selects the variable with the most integral relaxation value and tries that value first; MODEL uses the proposed network. Both run in the same loop: one warm-started relaxation per node (Section 4.2), identical propagation, identical value ordering, so the arms differ only in which variable they branch on and which value they try first. Each configuration is run three

times with different seeds; the depth-first search itself is deterministic given the relaxation vertex, so seeds change timing but not node counts.

Three points. First, the learned guidance produces a smaller tree at every size — $2.0\times$, $3.9\times$ and $9.5\times$ fewer nodes than LP — and the ratio grows with size, as the analysis of Section 4.9 predicts: the states where the network separates from relaxation rounding are the large, weakly constrained ones near the top of a big tree, and 10×25 has few of them. Second, a smaller tree buys time only once the tree is large enough to pay for the forward pass. At 10×25 the MODEL arm is *slower* (0.34 – $0.43\times$, faster on at most $1/30$ instances), because a 24 -node tree costs less than the network evaluations that shrink it; its per-node cost is 2.6 – $4.7\times$ that of LP at every size, which is the forward pass of Section 4.5. At 18×50 the median per-instance ratio is 1.30 – $1.37\times$ in the network’s favour but not significant (faster on $18/30$ in every seed, sign test $p = 0.36$). Third, at 21×60 the advantage is clear and replicates: the median per-instance ratio is $2.57\times$, $3.03\times$ and $2.96\times$ across seeds, the network is faster on exactly $23/30$ instances in each ($p = 0.005$), and both arms solve every instance within budget.

Two further strategies were measured on an earlier version of this loop that rebuilt the relaxation at every node. RANDOM branching solved $0/30$ at 20×50 within 300 s, and Section 4.12 shows it closes none of the 21×60 residual instances on the present loop either. LP-PROBE, which probes every free variable and soundly fixes the forced ones before branching as LP, produced by far the smallest trees ($51\times$ fewer nodes than LP at 20×50) and was the slowest arm, processing 6 nodes/s against 376 : node count and wall-clock rank the methods in opposite orders, and a tree-size convention would select the worst of them. We have not repeated LP-PROBE on the warm-started loop;

depth	% DET	network on DET	propagation	LP integrality	LP-probing	network on OPEN
5	61.9%	89.7%	1.2%	67.9%	51.6%	54.4%
6	74.2%	88.2%	1.6%	70.4%	68.0%	52.5%
7	79.9%	88.8%	3.7%	73.3%	72.8%	51.2%
8	87.9%	89.4%	5.3%	76.9%	79.8%	53.8%

Table 6: Residual decomposition on the same states as Table 4. Columns 4–6 are recall on the ground-truth DET set (LP-probing measured with the warm-started backend of Section 4.5); columns 3 and 7 are the network’s argmax accuracy on each subset.

Instances	n	network	LP-probing	ratio	probing recall
10×25	25	3.22 ms	5.60 ms	1.7×	—
18×50	50	2.33 ms	11.84 ms	5.1×	—
21×60	60	2.57 ms	18.39 ms	7.2×	—
40×100	100	4.02 ms	62.46 ms	15.5×	—
60×150	150	4.45 ms	79.06 ms	17.8×	—
<i>in-tree states of 10×25 instances, by depth:</i>					
depth 5	20	2.44 ms	4.02 ms	1.6×	51.6%
depth 6	19	2.39 ms	3.98 ms	1.7×	68.0%
depth 7	18	2.35 ms	3.43 ms	1.5×	72.8%
depth 8	17	2.34 ms	3.13 ms	1.3×	79.8%

Table 7: Per-node cost for a whole-instance verdict, warm-started LP backend for both methods, single-threaded. The network’s accuracy on DET variables at depths 5–8 is 85.9–92.7% against probing’s recall in the last column.

Section 4.5 measures its per-node cost there directly.

Replication on 100 instances. Thirty instances leave the sign test with little power and the median with wide uncertainty, so the 21×60 comparison was repeated on the 100-instance set (Table 9). The direction and significance hold — MODEL is faster on 70–71/100 instances ($p < 0.001$) with 5.5× fewer nodes — but the margins are smaller than on the original thirty: 2.0–2.3× in time against 2.6–3.0×, and 5.5× in nodes against 9.5×. The original thirty are the first thirty seeds of the larger set, and on them the supervised policy happens to do unusually well; we treat the 100-instance figures as the estimate and the 30-instance ones as the favourable end of its sampling variation.

We also record what the rebuilt-relaxation loop reported for the 21×60 comparison, because an earlier draft of this paper rested on it: 30/30 against 25/30 solved, and a 3.29× median speedup. Warm-starting the relaxation removed the solve-rate difference entirely — the five instances LP failed within 600 s it now closes in 65–270 s — and reduced the speedup. The solve-rate advantage was an artifact of a slow backend that penalized the arm expanding more nodes, and we do not claim it.

4.7. Comparison against complete solvers

Table 8 compares branching strategies inside our own search loop, which measures branching quality but says nothing about how the system stands against an off-the-shelf solver. That comparison is reported here, on the same instances, single-threaded, under the same budget.

The gap is large and we state it plainly: *the contribution of this paper is not a solver that competes with CP-SAT*. It is worth asking where the gap comes from, because the answer bears on

what the branching heuristic is actually doing. Table 11 decomposes it at 21×60 into tree size and per-node throughput, using CP-SAT’s own branch counter.

The learned guidance produces a *smaller* search tree than CP-SAT — 2.2× fewer nodes despite CP-SAT’s clause learning — and loses entirely on per-node cost. Where that cost goes can be measured. Before the loop was warm-started it processed 254 nodes/s, spending 4.34 ms per node rebuilding and solving the relaxation; warm-starting the same LP brings that to 0.08 ms and raises throughput to 619 nodes/s, which is the loop reported throughout Section 4.6. Of the 1.6 ms that remain per node, the network’s forward pass is 1.88 ms at the root and less at deeper nodes where the reduced graph is smaller, and propagation and bookkeeping are under 0.1 ms: the forward pass now accounts for over 90% of per-node cost, and we have not reduced it. We therefore do not claim that the gap is an implementation artifact. What the node counts establish is narrower: the deficit is not in branching quality. Closing it would require at minimum a compiled inference path and, as the next paragraph notes, algorithmic components our search lacks.

Two honest caveats attach to this reading. Tree sizes are not perfectly commensurable — CP-SAT’s “branches” include restarts and conflict-driven jumps, and each of its nodes performs more propagation work than ours — so the 2.2× should be read as an order-of-magnitude statement. And a competitive implementation would need more than a faster forward pass: clause learning and restarts are absent from our search, and the last two are what let CP-SAT scale past the sizes where we stop.

Size (budget)	strategy	solve rate	median nodes	median time	nodes/s
10×25 (60 s)	LP	$100 \pm 0.0\%$	46	0.03 ± 0.00 s	1,767
	MODEL	$100 \pm 0.0\%$	24	0.07 ± 0.01 s	376
18×50 (300 s)	LP	$100 \pm 0.0\%$	4,224	2.43 ± 0.33 s	2,066
	MODEL	$100 \pm 0.0\%$	1,085	1.82 ± 0.37 s	731
21×60 (600 s)	LP	$100 \pm 0.0\%$	82,366	51.5 ± 2.0 s	1,627
	MODEL	$100 \pm 0.0\%$	8,657	14.6 ± 0.7 s	619

Table 8: Branching strategy comparison on the warm-started loop, lattice stage disabled, 30 instances per cell, mean $\pm$ standard deviation over three seeds of the per-seed median. Node counts are seed-invariant.

strategy	solve rate	median nodes	median time	nodes/s	vs. LP (per instance)
LP	$100 \pm 0.0\%$	79,609	41.2 ± 1.9 s	1,966	—
MODEL	$100 \pm 0.0\%$	13,386	16.5 ± 0.6 s	796	2.0–2.3 $\times$, 70–71/100, $p < 0.001$
MODEL+FT (Section 4.11)	$100 \pm 0.0\%$	5,495	7.6 ± 0.3 s	738	4.8–5.9 $\times$, 81–84/100, $p < 0.001$

Table 9: 21×60 , 100 instances, lattice off, 600 s budget, three seeds (mean $\pm$ s.d. of per-seed medians; node counts are seed-invariant). The last row is the cost-aware fine-tuned policy of Section 4.11.

4.8. Size transfer

Because the features are size-normalized and the parameters do not depend on (m, n) , one network applies at any size. We test whether that transfers in practice, with $|S|$ matched as in Table 1.

At the 21×60 evaluation size the network trained only at 10×25 — a 2.4 $\times$ smaller problem — is not worse than the one trained at 18×50 ; if anything it is better, expanding fewer nodes on 18/30 instances and finishing faster on 18–19/30 in each seed, with a median per-instance time ratio of 1.24–1.31 $\times$ in its favour. Neither difference is significant (sign test $p \geq 0.20$), and both solve 30/30. At 18×50 the picture is the same: the 10×25 network is faster on 16–17/30 instances ($p \geq 0.59$) although its median time is slightly higher, the two summaries disagreeing because the paired differences are small against instance-to-instance variation. A previous version of this comparison, on the rebuilt-relaxation loop, gave 15/30 and a 0.92 $\times$ ratio; the direction is not stable across loops and we read the result as no detectable transfer loss rather than as a transfer gain. Root-level prediction agrees: the 10×25 network scores 67.8%, 69.5% and 69.4% at 10×25 , 20×50 and 40×100 , holding its margin over relaxation rounding (+11.0, +7.2, +4.7 points) at every size.

This matters practically because label generation is the binding constraint. Exact marginals require enumeration, which fails beyond $n \approx 60$; transfer means a network trained where labels are obtainable can be deployed where they are not.

4.9. Where the transferred network helps

Section 4.4 located the useful prediction regime at depths 6–8 of a 10×25 instance. Whether that localization survives transfer is a separate question, and it does not: at 21×60 the search queries the network on states with 5–60 free variables, only 19.7% of which fall inside the training range of 13–25, and 80.2% of which are larger than any training state. Table 13 evaluates the network and relaxation rounding against exact conditional marginals on 21×60 in-tree states, grouped by the number

of free variables, on the subset of states that could be enumerated within 15 s.

The pattern is the reverse of what a depth-based account predicts. Inside the training range the network is indistinguishable from relaxation rounding — both are perfect and their outputs correlate at 1.000 — because at $m/|K| \geq 0.8$ the reduced instance is over-constrained and the relaxation is already integral on every DET variable. The network’s entire margin arises on states it never saw during training, at 50–60 free variables, where the relaxation is weakest ($m/|K| \approx 0.35$ –0.42) and its outputs have decorrelated from the relaxation’s. In terms of Section 3.4, this is the region where DET variables are present but the relaxation does not find them.

Two conclusions follow. The first corrects Section 4.4: depth was a proxy for constraint density that happened to hold on 10×25 , where descending from the root moves $m/|K|$ only from 0.40 to 0.59, and does not hold on 21×60 , where the same descent moves it from 0.35 past 1.0. The useful regime is defined by $m/|K|$, and at deployment size it sits near the *top* of the tree. The second is that the transfer result of Section 4.8 is stronger than a matched-multiplicity comparison alone shows: the network extrapolates to states outside its training range in both size and constraint density, and that extrapolation is where its contribution lives. It is also a limitation, since the training range was not chosen with this in mind (Section 4.14).

4.10. Training on deployment-sized states: a negative result

Section 4.9 located the network’s margin on states with 50–60 free variables, outside the 13–25 range of its training states, and Section 4.14 of an earlier draft listed aligning the two as an obvious untested improvement. We tested it, holding everything but the training-state distribution fixed.

States.. From a separate 21×60 pool (Section 4.2) we built in-tree states with 30, 35, $\dots$, 60 free variables by fixing a prefix of the LP-confidence ordering to the planted solution, and

Size	CP-SAT	SCIP	proposed (MODEL)	CP-SAT advantage
10×25	0.002 s	0.009 s	0.07 s	35×
18×50	0.046 s	0.385 s	1.82 s	40×
21×60	1.159 s	3.687 s	14.6 s	13×
21×60 , 100 inst.	0.792 s	2.590 s	16.5 s / 7.6 s (fine-tuned)	21× / 10×

Table 10: Median solve time; all methods solve every instance at every size. The proposed column is the MODEL arm of Table 8 (mean of per-seed medians); the last row adds the 100-instance set and the fine-tuned policy of Section 4.11. The proposed system does not beat a complete solver at any size tested, and is outperformed by SCIP, whose LP code it uses for training features.

	CP-SAT	proposed	ratio
nodes (median)	18,827 branches	8,657	ours 2.2× fewer
throughput	19,782 nodes/s	619 nodes/s	CP-SAT 32× faster
total time (median)	1.065 s	14.6 s	14×

Table 11: Decomposition of the 21×60 gap. CP-SAT additionally learns a median of 5,514 conflict clauses per instance.

labelled them with exact conditional marginals under the enumeration rule of Section 3.5. Every state with at most 55 free variables enumerated within 90 s; at 60 (the root) 291/400 did, and the failures are the instances with large $|S|$, so root labels are biased toward small solution sets. A property of these states matters for what follows: with the prefix fixed to a solution, the surviving solution set is almost always a singleton once 50 or fewer variables remain (median $|S(F, \mathbf{v})| = 1$, DET share 97%), so their labels are nearly hard. Three networks were trained with the architecture, optimizer, epochs and loss of Section 4.2: M0, the network used everywhere above (10×25 in-tree states); M1, the 2,691 deployment-range states only; M2, half of each. A pre-diagnostic on 10×25 alone, training on depth ranges 0–6, 6–12 and 0–12 with equal state counts, gave in-tree accuracies of 79.3%, 79.9% and 79.9%: within one size, the depth range does not matter.

Prediction.. Re-running the analysis of Table 13 with the three networks scored on identical states, the gain over relaxation rounding on DET variables at 50 and 60 free variables is +5.4 and +9.7 points for M0, +6.1 and +10.7 for M1, +5.9 and +10.6 for M2. Training in the deployment regime buys about one point there.

Search.. It costs far more elsewhere. On the 100-instance set M1 expands a median of 21,337 nodes against 13,386 for M0 (1.6× more; fewer on only 38/100, $p = 0.021$) and is 1.7–1.8× slower; on the original thirty it expands 20,746 against 8,657. M2 sits between (18,558, $p = 0.089$). A switch that routes nodes with ≥ 50 free variables to M1 and the rest to M0 recovers nothing (17,252 nodes, 43/99, $p = 0.23$). The mechanism is visible in the prediction diagnostics: on small, dense interior states (10×25 at depth 6–8, the kind the search spends two thirds of its nodes on) the probability that M1’s most confident variable is DET and predicted correctly falls from 94.4% to 77.6%, and the rank correlation between its confidence and the true margin from 0.36 to 0.12, while on the deployment-range states it improves (77.2 $\rightarrow$ 86.0%). Nodes are where the interior is, and a network that is slightly better near the top of the

tree and much worse inside it makes the search slower. We conclude that the training-state size distribution is not the lever, and that the residual identified in Section 4.9 is not a distribution-shift artifact.

4.11. Cost-aware fine-tuning

If neither accuracy on DET variables nor the training distribution explains what the supervised guidance leaves on the table, the natural suspect is the objective: the rule branches on the variable it is surest about, and being sure is not the same as being cheap. Three measurements test this, and the third turns it into a method.

R0: does the confident choice cost the least?. On 100 held-out states with 45–50 free variables we forced the root decision to each of the five variables the network is most confident about, ran the supervised search below it, and recorded the exact subtree cost of each. The confidence rule’s choice is the cheapest of the five on 22% of states; the median ratio of its cost to the cheapest candidate is 1.29 and the mean 2.97; on 24% of states some candidate is at least twice as cheap; and choosing the cheapest candidate at the root alone would remove 55% of all nodes summed over these states. The LP rule’s root choice costs the same as the network’s on only 10% of states, so the two rules are not merely tied. There is a large residual, and it is about *which* variable to branch on.

R1: can the cost be predicted?. The critic is the baseline of the policy update, so it must rank decisions by cost on states it has not seen. Recording every decision of the supervised search on the training pool gave 37,275 decisions (9,375 on held-out states) with their exact subtree costs, of which two thirds occur at 25–34 free variables. A value head on the frozen trunk reaches a held-out Spearman correlation of 0.80 with $\log c$; a baseline that knows only the state’s size (a linear fit on $\log |K|$) reaches 0.70, and within size buckets the critic’s advantage is larger (0.68 against 0.38 at 35–44 free variables, 0.57 against 0.37 at 45–55). Unfreezing the trunk adds 0.02. The gate of Section 3.8 is passed with the frozen critic.

R2: fine-tuning.. Six epochs of 400 rollouts each ($\tau = 1$, 90 s budget, about 21,000 decisions per epoch) from the 45–60 free-variable training states. On the 150 held-out monitor states the total subtree cost under the greedy policy, relative to the supervised rule, was 0.79, 0.49, 0.56, 0.46, 0.65 and 0.56 after epochs 1–6; epoch 4 was selected. Policy entropy fell from 1.73 to 0.10

Training size	Evaluation size	solve rate	median time	median nodes
10 × 25	10 × 25	100%	0.07 ± 0.01 s	24
10 × 25	18 × 50	100%	1.82 ± 0.37 s	1,085
18 × 50	18 × 50	100%	1.48 ± 0.15 s	942
10 × 25	21 × 60	100%	14.6 ± 0.7 s	8,657
18 × 50	21 × 60	100%	23.7 ± 0.7 s	14,742

Table 12: Transfer on the warm-started loop, lattice stage disabled, 30 instances per row, three seeds. Rows 2–3 and 4–5 each hold the evaluation size fixed and vary only the training size.

free vars	in training range	$m/ K $	network on DET	LP on DET	gain	corr(network, LP)
20	yes	1.05	100.0%	100.0%	+0.0	1.000
25	yes	0.84	100.0%	100.0%	+0.0	0.998
30	no	0.70	95.7%	95.0%	+0.7	0.954
40	no	0.53	75.5%	75.2%	+0.4	0.747
50	no	0.42	70.2%	65.0%	+5.3	0.664
60	no	0.35	93.3%	88.9%	+4.4	0.658

Table 13: 21 × 60 in-tree states by number of free variables, 9–14 enumerable states per row. Gain is the network’s margin over relaxation rounding on DET variables; the last column is the Pearson correlation between the two predictors’ outputs.

over the same epochs, so the policy is near-deterministic by the selected epoch.

Evaluation.. Table 9 reports the fine-tuned policy (MODEL+FT) beside the supervised one on the 100-instance set: 5,495 nodes against 13,386 (2.3× fewer, fewer on 70/100, $p < 0.001$ in every seed) and a median of 7.6 s against 16.5 s (2.0–2.2×, faster on 64–67/100, $p \leq 0.007$), at an unchanged per-node cost (738 against 796 nodes/s). Against LP-guided branching this is 4.8–5.9× in time and 14× in nodes. On the original thirty instances the effect is small (8,873 against 8,657 nodes; 1.2–1.5× in time, 18–21/30, p between 0.04 and 0.36): the seventy added instances account for the gain (2.5× fewer nodes, 53/70), which is consistent with the thirty being a favourable sample for the supervised rule. In the deployed cascade (Table 14) the fine-tuned policy closes the lattice residual in 7.3–11.7 s against 12.2–17.7 s for the supervised one.

Replication and transfer of the fine-tuned policy.. A second fine-tuning run with a different training seed (different rollout sampling and initialization of the correction head; epoch 2 selected by the same monitor) gives the same picture on the 100-instance set — 4,940 nodes (2.4× fewer than the supervised policy, fewer on 68/100, $p < 0.001$) and 1.9–2.4× faster (66–68/100, $p \leq 0.002$) — and a larger one on the original thirty (4,040 nodes, 1.9× fewer on 19/30; 2.3× faster, 20–21/30). The two runs select different variables at many nodes and reach similar costs, which is what one expects if the objective, not a particular solution, is what was learned. The policy was fine-tuned on 21 × 60 states only, yet at 18 × 50 it expands 424 nodes against the supervised policy’s 1,085 (1.7× fewer, 20/30, $p = 0.099$) and is 1.8–2.2× faster (22–24/30, $p \leq 0.016$): the cost-aware improvement transfers downward in size as the supervised one does.

What changed.. On 300 held-out states the fine-tuned policy branches on a different root variable than the confidence rule at 95% of them. Its pick is typically the 15th most confident variable ($|\hat{p} - \frac{1}{2}|$ of 0.26 against 0.43 for the rule’s pick), and it is DET with a correct first value less often (57.5% against 82.5%). The policy has moved *away* from the variables it is surest about, and the search got cheaper. The improvement is not attributable to the root decision alone: forcing only the root to the fine-tuned pick and running the supervised policy below it gives a median cost ratio of 0.93 (cheaper on 22/46 disagreement states), so the gain is a property of the sequence of decisions rather than of any single one. We can therefore say what the fine-tuning optimizes — subtree cost, with the probability of being right held as a soft prior — but not yet name the feature it found; a cheap proxy, the amount of propagation the first child triggers, does not differ between the two choices.

4.12. Deployment in the full pipeline

Table 8 isolates branching by disabling lattice reduction. In deployment the lattice stage runs first and the search sees only its residual. Because that stage is independent of the branching heuristic, every arm receives the identical residual set.

Lattice coverage falls with size — 30/30, 24–28/30, 10–16/30 — and that decline is what creates room for branching quality to matter. At 10 × 25 and 18 × 50 the residual is empty or closed within seconds by either sound strategy. At 21 × 60 between ten and twenty instances reach the search depending on the seed, and the residual is genuinely hard: random branching closes none of the 50 residual instances pooled over seeds, so the end-to-end rate of that arm is the lattice coverage alone. Both relaxation-based and learned branching close all 50, and the difference between them is time. On the residual the learned guidance is faster on 11/14, 14/16 and 15/20 instances in the three seeds (median per-instance ratio 3.48×, 2.99×, 2.38×;

Size	strategy	closed by lattice	residual solved by search	median time on residual	end-to-end
10 × 25	both	30/30	—	—	100%
18 × 50	LP / MODEL	24–28/30	11/11 / 11/11	0.5–5.0 s / 0.6–2.5 s	100% / 100%
21 × 60	RANDOM	10–16/30	0/50	> 600 s	44.4 ± 8.3%
21 × 60	LP	10–16/30	50/50	52.9–65.1 s	100 ± 0.0%
21 × 60	MODEL	10–16/30	50/50	14.2–20.2 s	100 ± 0.0%
21 × 60	MODEL+FT	10–16/30	50/50	7.3–11.7 s	100 ± 0.0%

Table 14: Cascade results on the warm-started loop, three seeds at every size, 600 s budget at 21 × 60. Lattice coverage varies across seeds (16, 14, 10 of 30 at 21 × 60) because the lattice stage draws random column permutations, so the residual handed to the search varies with it; all arms within a seed receive the identical residual.

Model	accuracy	L_1 to marginal	parameters
Linear, variable features only	78.1%	0.2530	4
MLP, variable features only	77.8%	0.2522	8,641
MLP + constraint aggregates	79.4%	0.2361	9,025
MarginalNet (bipartite graph)	84.1%	0.1514	21,761
Bayes ceiling	87.3%	0.0000	—

Table 15: Ablation on in-tree states of 10 × 25 instances, 2,000 training and 800 test states.

sign test $p = 0.057, 0.004, 0.041$), 40/50 pooled with a median ratio of $3.00 \times (p = 2.4 \times 10^{-5})$, expanding 6.9–9.5× fewer nodes. The residual is where the branching heuristic operates in deployment, and the speedup there matches the isolated measurement of Table 8.

4.13. Does the graph structure contribute?

MarginalNet consumes six scalars per node and propagates them over the bipartite graph. With a feature set that small, a per-variable model ignoring the graph is a serious baseline: if it matches, the message passing is decoration. Table 15 compares models trained on identical targets, loss and splits, differing only in whether structure is used. The aggregated variant appends mean and max of the constraint-side features over each variable’s incident rows, which is the information one message-passing round would deliver.

Message passing contributes: +4.7 points over the best structure-free model and a 36% reduction in L_1 , closing roughly 60% of the residual gap to the ceiling that the MLP leaves open. The improvement in L_1 is the more relevant of the two, since the analysis of Section 3.4 consumes the marginal rather than its argmax.

The same table bounds the claim, though. A four-parameter linear model on the relaxation value alone reaches 78.1%, within 9.2 points of the ceiling; most of the available signal is in that single feature, and the graph adds a useful but not dominant increment on top of it.

4.14. Limitations

Label generation bounds the method, not only its evaluation.. Exact marginals require enumerating $\mathcal{S}$, which does not complete beyond $n \approx 60$ on this family. Transfer permits deployment at larger sizes but not training there; approximating p_j by solution sampling is the natural extension and is untested.

The useful size window is bounded on both sides.. Below it, lattice reduction closes everything and branching quality is irrelevant. Above it ($n = 100$), search fails outright for every strategy, since a constraint solver finds no solution within 20 s. Our demonstration lives at $n = 60$ and we do not show the window extends further.

The advantage is in time and tree size, not in solve rate.. On a warm-started loop both sound strategies solve every instance we evaluate within budget; what the supervised guidance buys at 21 × 60 is a 2.0–2.3× median speedup and 5.5× fewer nodes on 100 instances (2.6–3.0× and 9.5× on the original thirty, a favourable sample), and the fine-tuned policy a further 2.0–2.2× and 2.3×, significant per instance in every seed. An earlier draft reported a 5–0 solve-rate advantage from a loop that rebuilt the relaxation at every node; that difference vanished with the faster backend and we do not claim it. Whether a solve-rate advantage exists at larger budgets or sizes is untested, and with 30 evaluation instances per size the power to detect one would be low in any case.

Multiplicity control is approximate.. Median $|\mathcal{S}|$ is 23, 19 and 10 across the three sizes; at $n = 60$ multiplicity steps $42 \rightarrow 12 \rightarrow 2$ across consecutive integer m , so finer matching is unavailable. Three of thirty evaluation instances have unverified $|\mathcal{S}|$ and are marked as such. Matching $|\mathcal{S}|$ also moves m/n from 0.40 to 0.35.

Cost advantage depends on scale and on baseline engineering.. The amortization argument holds asymptotically but is worth little at the sizes where our search operates: 1.3–1.7× at the depths where prediction matters, against a warm-started probing baseline that is additionally sound. It reaches 15–18× only at $n \geq 100$, where search fails for every strategy we tested. We therefore cannot demonstrate the regime in which the argument is strongest.

The search implementation is not competitive.. At 21 × 60 CP-SAT is 9–10× faster than the fine-tuned policy and 20× faster than the supervised one on the 100-instance set (13× on the original thirty). Our branching produces fewer nodes, so the deficit is per-node throughput (32×), now dominated by the network’s forward pass; we have not shown that a competitive implementation is achievable: a compiled inference path, clause learning and restarts are all absent, and the latter two are load-bearing for CP-SAT at larger sizes.

Training and deployment distributions barely overlap, and aligning them did not help. The training range $\{0, \dots, 12\}$ produces states with 13–25 free variables, whereas the 21×60 search queries states with 5–60 and spends 80% of its queries above 25. Section 4.10 shows that training on the deployment regime makes the search slower; we do not know whether a different construction of large training states (prefixes from the search’s own trajectory rather than from a solution, or states with several surviving solutions) would behave differently.

The fine-tuning result rests on one training pool and few training seeds. The policy was fine-tuned on states from one 21×60 pool; two training seeds agree at 21×60 and the gain transfers to 18×50 , but no larger size was tested and both seeds select their epoch on the same monitor. Its entropy collapses within four epochs, epoch selection is by a held-out monitor rather than by any principled stopping rule, and we cannot yet name the feature the policy exploits (Section 4.11). Whether the gain transfers across sizes as the supervised one does is untested.

Hyperparameters are untuned and there is no validation split for the supervised network. See Section 4.2. The configuration is defensible as a first attempt and is not contaminated by the test set; the depth range, at least, was subsequently shown not to matter within a size (Section 4.10). The fine-tuning stage does select an epoch on held-out states, which are drawn from the training pool and disjoint from every evaluation instance.

Single instance family. All results concern one generator from one application. Whether the amortization argument transfers to other constraint families is untested.

5. Conclusion

We presented a learned branching heuristic for binary linear systems together with an exact procedure for supervising it. Two elements carry the method. Conditioning a BLS on a partial assignment is identical to reducing it, so a single network trained on ordinary instances predicts at every node of the search tree without architectural change. And on instances small enough to enumerate, the exact conditional posterior both supplies the training target and identifies which predictions can affect the search at all: only variables that every surviving solution agrees on can cost a backtrack, and their share rises from 6.3% at the root to 87.9% at depth 8.

Measured inside our search loop the heuristic does what it is designed to do. It is 2.0–2.3× faster in median than relaxation-based branching at 21×60 on 100 instances in each of three seeds, expands 5.5× fewer nodes, and closes the residual of the deployed pipeline 3.0× faster, where random branching closes nothing; with solution multiplicity controlled, a network trained at 2.4× smaller scale performs indistinguishably from one trained at the evaluation size, which matters because exact supervision is available only at small sizes.

We are equally explicit about what the results do not establish. The system does not compete with a complete solver: CP-SAT is 9–40× faster at every size we tested. Decomposing

that gap is informative rather than exculpatory — our branching yields 2.2× fewer nodes and loses on throughput by 32×, which places the deficit in a Python research implementation whose per-node cost is now the network’s forward pass, and which lacks clause learning and restarts — but we have not shown that closing it is achievable. Nor does the cost argument carry the weight we initially assigned it. Against a properly warm-started probing baseline the advantage is 1.3–1.7× at the depths where prediction matters on our instances, and probing is sound where the network is not; the ratio reaches 15–18× only at $n \geq 100$, a regime in which search fails for every strategy we tried. An earlier version of this comparison, using a backend that rebuilt the LP for each probe and excluded the network’s own feature solve from its timer, reported 19–23× at a scale where the true figure is near unity, and we record the correction because the error came entirely from instrumentation rather than from what the two methods compute.

Two further results sharpen what the supervised target is. Moving the training states to the deployment regime, where the network’s prediction margin lives, does not help and in fact doubles the search tree, because the search spends its nodes in the small interior states the moved network no longer handles well; the training-state distribution is not the lever. Optimizing the cost the search pays is. With the exact subtree cost of every decision read off the search, an actor-critic fine-tuning of the branching choice — initialized at the supervised rule, with the marginal network frozen — expands 2.3× fewer nodes and runs 2.0–2.2× faster than the supervised policy on 100 instances, and the policy it finds branches on variables the network is *less* sure about. The probability that a branch is right and the cost of the branch being wrong are different objectives, and the second is the one a search minimizes.

What remains, and what we claim, is methodological. For planted-solution instance families the exact conditional posterior is computable and is a well-posed supervision signal for branching, where the conventional target — a single planted solution — is not. States drawn from inside the search tree are a better training distribution than root states, and the reduction identity makes generating them a matter of substitution rather than solver instrumentation. The resulting heuristic transfers across instance size when solution multiplicity is held comparable, and the exact cost of its decisions, available from the search, is a second well-posed training signal that improves it. Whether this becomes practically useful depends on two things we have not done: extending exact supervision past $n \approx 60$, most plausibly by approximating the marginals without exhaustive enumeration, and implementing the search well enough that a branching improvement is visible at the system level.

Reproducibility

Code is organized by component under `proposed_src/`, with shared modules in `util/` and a `ROLES.txt` in each directory describing every file. Scripts are run from their own directory. Every search number in Tables 8, 12 and 14 comes from one chain, `v23_ablation/run_revision2.sh`, which

runs the warm-started loop (v23_warm_search.py) over all conditions and seeds; v23_aggregate.py produces the tables from its output.

```
# instances with matched |S| and exact marginals
cd proposed_src/v15_bayes
python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
--out ../../runs/v22/sols_10x25.json

# attach solution sets, then split 80/20 (seed 0)
cd ../v22_transfer
python3 v22_add_solutions.py \
--inp ../../runs/v22/sols_10x25.json \
--out ../../runs/v22/full_10x25.json
python3 v22_split.py # -> full_10x25_{train,test}

# in-tree training states (depths 0-12, 6 per instance)
cd ../v17_conditional
python3 v17_make_conditional_data.py \
--sols ../../runs/v22/full_10x25_train.json \
--max_depth 12 --per_instance 6 \
--out ../../runs/v22/cond_10x25_train.json

# train (about 20 minutes, single CPU core)
python3 v17_train_conditional.py \
--train ../../runs/v22/cond_10x25_train.json \
--test ../../runs/v22/cond_10x25_test.json \
--n_train 10000 --n_test 2000 --epochs 20 \
--target soft --out ../../runs/model_n25

# every search table: warm-started loop, 3 seeds,
# conditions A/B/R/C/D/Crand x {lattice off, on}
cd ../v23_ablation
bash run_revision2.sh # -> runs/v23/, chain.log
python3 v23_aggregate.py # -> runs/v23/aggregate.json

# v24: range training (negative), 100-instance
# set, R0/R1/R2 fine-tuning and evaluation
cd ../v24_deploy_range
bash run_v24.sh; bash run_v24_switch.sh
bash run_v24_r1.sh; bash run_v24_r2.sh
python3 v24_aggregate.py \
--tags M0 M1 M2 SW PV
```

All generators are seeded. The released checkpoints are 96 KB each. Instance data and weights are regenerated from the above rather than distributed, and the full evaluation cycle is scripted in run_revision2.sh under v23_ablation/.

References

- [1] Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- [2] Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- [3] Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- [4] Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421.
- [5] Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- [6] Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- [7] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*.
- [8] Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of the Cognitive Science Society*, pages 517–522.
- [9] Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*, pages 724–731.
- [10] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- [11] Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- [12] Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- [13] Millidge, B. (2022). Deconfusing direct vs amortized optimization. Unrefereed technical note, <https://www.bereni.io/>, post 2022-09-25-Deconfusing-direct-vs-amortized-optimization.
- [14] Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *Design Automation Conference*, pages 530–535.
- [15] Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- [16] Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.

- [17] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- [18] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.
- [19] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.
- [20] Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable selection in a branch and bound algorithm. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, pages 176–185.
- [21] Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In *Advances in Neural Information Processing Systems*, volume 35.
- [22] Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.
- [23] Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In *AAAI Conference on Artificial Intelligence*.
- [24] Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In *CPAIOR*.
- [25] Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In *AAAI Conference on Artificial Intelligence*.
- [26] Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In *International Conference on Learning Representations*.
- [27] Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI Conference on Artificial Intelligence*.
- [28] Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In *Advances in Neural Information Processing Systems*, volume 35.
- [29] Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In *Allerton Conference on Communication, Control, and Computing*.
- [30] Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. *arXiv:2012.13349*.
- [31] Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In *AAAI Conference on Artificial Intelligence*.
- [32] Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. *Journal of Artificial Intelligence Research*, 43:173–210.
- [33] Refalo, P. (2004). Impact-based search strategies for constraint programming. In *Principles and Practice of Constraint Programming (CP)*, pages 557–571.
- [34] Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In *AAAI Conference on Artificial Intelligence*.
- [35] Wassermann, A. (2025). Solving the market split problem with lattice enumeration. *arXiv:2508.08702*; to appear in *Mathematical Programming Computation*.
- [36] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In *International Conference on Learning Representations*.